

Claude Fit Logger

Installation Guide (Personal Use)

1 Overview

This guide walks through a from-scratch personal installation of Claude Fit Logger: a custom MCP server that lets Claude log food/drink directly to Google Health, with a Neon Postgres store tracking entries for undo/remove.

This system is designed for a **single user**. It is not intended to be deployed for multiple accounts without further changes (see the Security Guide for why).

2 Prerequisites

- A Google account
- Python 3.11+ installed locally, with `pip`
- A GitHub account (for hosting the server code and triggering deploys)
- Accounts (all free tier): Render.com, Neon.tech
- A free USDA FoodData Central API key (signup link)

3 Step 1: Google Cloud Project

1. Create a project at Google Cloud Console.
2. Enable the **Google Health API** for the project.
3. Under *Google Auth Platform* → *Branding*, complete the OAuth consent screen setup (External user type).
4. Under *Data Access*, add these two scopes:

```
https://www.googleapis.com/auth/googlehealth.nutrition.readonly  
https://www.googleapis.com/auth/googlehealth.nutrition.writeonly
```

5. Under *Audience*, add yourself as a test user, then click **Publish App** (moves status to Production — required so your refresh token does not expire after 7 days).
6. Under *Clients*, create an OAuth client of type **Desktop app**. Download the resulting `client_secret.json`.

4 Step 2: Obtain a Refresh Token

Locally, install the OAuth helper library and run the consent script once:

```
pip install google-auth-oauthlib --break-system-packages  
python get_refresh_token_health.py
```

This opens a browser consent screen. Approve it. The script prints a **refresh token**, **client ID**, and **client secret** — save all three somewhere private (never commit them to git).

5 Step 3: Neon (Postgres)

1. Create a free project at neon.tech.
2. Copy the connection string shown after project creation (format: `postgresql://user:pass@host/db?sslmode=...`).

6 Step 4: Server Code

1. Clone or create a local folder containing: `main.py`, `db.py`, `health_client.py`, `nutrition_lookup.py`, `oauth.py`, `requirements.txt`.
2. Push this folder to a new (private is fine) GitHub repository.
3. Add a `.gitignore` excluding: `.venv`, `client_secret.json`, any file containing real credentials, and `__pycache__`.

7 Step 5: Deploy to Render

1. New Web Service → connect the GitHub repo.
2. **Root directory:** `server`
3. **Build command:** `pip install -r requirements.txt`
4. **Start command:** `python main.py`
5. **Instance type:** Free
6. **Health check path:** `/health`
7. Add environment variables (see table below).
8. Deploy. Confirm `https://your-service.onrender.com/health` returns ok.

Variable	Source
<code>MCP_API_KEY</code>	Self-generated random secret
<code>PUBLIC_BASE_URL</code>	Your Render service URL
<code>DATABASE_URL</code>	Neon connection string
<code>GOOGLE_CLIENT_ID</code>	Step 1
<code>GOOGLE_CLIENT_SECRET</code>	Step 1
<code>GOOGLE_REFRESH_TOKEN</code>	Step 2
<code>USDA_API_KEY</code>	FoodData Central signup

8 Step 6: Connect to Claude

1. In Claude: *Settings* → *Connectors* → *Add custom connector*.
2. Name it, and set the URL to your Render service URL (no query parameters needed — auth happens via the built-in OAuth shim).
3. Leave OAuth Client ID/Secret blank; click **Connect**.

9 Step 7: Update the Skill

Install/update the `calorie-counter` skill in Claude's Skill settings so it calls `log_food`, `undo_entry`, and `remove_entry` instead of doing its own separate nutrition lookup.

10 Verification

In a new chat, say: *“Log 150g of grilled chicken breast.”* Confirm the entry appears both in the chat table and in the Google Health app under today's nutrition.